Titan Core Engine Map (Master Blueprint)

Purpose

This document defines the directional relationships between Titan core engines.

It is the canonical map for:

where responsibilities live

which engine is allowed to write to which tables

how signals, automation, assistants, and governance interact

Engine Roles at a Glance

Titan Identity Engine → who you are (auth, roles, tenancy)
Titan Customer Engine → who/where the work is for (customers, sites)
Titan Work Engine → what work exists + lifecycle state (jobs, tasks, visits)
Titan Evidence Engine → proof + compliance (photos, signatures, chain)
Titan Docs Engine → documents + bundles + versions (reports, contracts, packs)
Titan Money Engine → quotes → invoices → payments (financial lifecycle)
Titan Talk Engine → communications + threads (omnichannel inbox)
Titan Signal Engine → append-only event bus (what happened)
Titan Pulse Engine → automation rules (what to do next)
Titan Zero Engine → AI reasoning + governance (approve/plan/route models)
Titan Analytics Engine → metrics + insights (consumes signals)

The Titan Execution Spine (Primary Flow)

User / UI Action
↓
Core Engine API (Work / Money / Talk / Customer / Evidence / Docs)
↓
tz_* tables updated (state + satellites)
↓
Titan Signal Engine emits event (append-only)

↓
Titan Pulse Engine evaluates rules
↓
Pulse executes actions through Core Engine APIs
↓
Titan Signal emits downstream events
↓
Analytics consumes signals → metrics → dashboards

Key rule: Core engines write operational state.
Signal records facts. Pulse orchestrates. Analytics observes.

Governance Overlay (Risk Gated Flow)

Signal or Pulse Action Proposal
↓
Risk Classifier (LOW / MEDIUM / HIGH / CRITICAL)
↓
LOW → execute automatically (Pulse → Engine API)
MEDIUM → optional confirmation / policy check
HIGH → Titan Zero Engine approval required
CRITICAL → Titan Zero + human confirmation required

Titan Zero decision loop:

OmegaCore (strategy) + OmicronCore (detail validation) + TitanZeroCore (synthesis)
→ Trinary Bayesian Mode
→ approve / modify / reject / defer

Key rule: Titan Zero proposes and approves. It does not write directly to tz_ tables.

Interfaces to Engines Mapping

Titan Command (single page business control panel)

Primary surfaces:

signals requiring action

risk + approvals

automation health

dispatch visibility

money at risk

Command → reads: Analytics + Signals + Engine summaries
Command → writes: approvals, overrides, dispatch decisions via Engine APIs

Titan Talk (comms interface)

Talk UI → Talk Engine → tz_threads / tz_messages
Talk Engine → emits signals: customer.message.received / message.sent
Pulse / Assistants react to communication signals

Titan Origin (traditional admin screens)

Origin UI → direct CRUD via Engine APIs (not raw table writes)
Purpose: users who prefer classic admin screens

Titan Go (mobile/PWA interface with modes)

Modes:

Field Mode

Command Mode

Dispatch Mode

QC Mode

Titan Go never bypasses engines; it calls the same Engine APIs with simplified UI.

Titan Go Mode → Engine Dependencies

Field Mode

Work Engine (jobs/visits/tasks/checklists)
Evidence Engine (capture proof)
Talk Engine (job-linked messages)
Signal Engine (job.started/job.completed/evidence.uploaded)

Command Mode

Analytics Engine (dashboards)
Signal Engine (inbox alerts)
Pulse Engine (automation status)
Zero Engine (approval queue)
Work/Money summaries via Engine APIs

Dispatch Mode

Work Engine (assignments, schedules, visits)
Talk Engine (notify workers/customers)
Signal Engine (assignment changes)
Pulse Engine (dispatch automations)

QC Mode

Evidence Engine (inspection proof)
Work Engine (QC outcomes tied to job lifecycle)
Docs Engine (inspection reports/bundles)
Signal Engine (qc.approved/qc.failed)
Pulse Engine (rework workflows)
Zero Engine (high-risk dispute escalation)

Core Engine Write Boundaries (Non-Negotiable)

Only these engines write their domains:

Work Engine → tz_jobs*, tz_tasks*, tz_visits*, tz_checklists*

Customer Engine → tz_customers*, tz_sites*

Money Engine → tz_quotes*, tz_invoices*, tz_payments*, tz_financial_events*

Talk Engine → tz_threads*, tz_messages*, tz_contacts*

Evidence Engine → tz_evidence_*

Docs Engine → tz_documents*

Signal Engine writes only:

tz_signals* (append-only)

Pulse Engine writes only:

tz_automation_* plus it executes actions through Engine APIs

Zero Engine writes only:

governance + AI artifacts/logs (never operational tz_ state)

Analytics Engine writes only:

tz_metrics*, tz_metric_snapshots*

Engine Dependency Graph (Directional)

Identity Engine
  → all engines (auth + tenant boundary)

Customer Engine
  → Work Engine (jobs reference customers/sites)
  → Money Engine (quotes/invoices reference customers)

Work Engine
  → Evidence Engine (evidence linked to jobs/visits)
  → Docs Engine (reports/bundles for jobs/QC)
  → Talk Engine (threads linked to jobs)

Money Engine
→ Talk Engine (invoice messages/reminders)
→ Docs Engine (quotes/contracts as documents)

Talk Engine
→ Signal Engine (message events)

Evidence Engine
→ Signal Engine (evidence uploaded)

Docs Engine
→ Signal Engine (document created/bundled)

Work/Money/Talk/Evidence/Docs/Customer
→ Signal Engine (emit lifecycle signals)

Signal Engine
→ Pulse Engine (automation triggers)
→ Zero Engine (governance triggers + assistant activation)
→ Analytics Engine (metrics ingestion)

Pulse Engine
→ Core Engines (execute actions via APIs)
→ Signal Engine (emit automation.* outcomes)

Zero Engine
→ Core Engines (propose actions; approved actions executed via APIs)
→ Talk Engine (draft messages)
→ Docs Engine (generate artifacts)
→ Signal Engine (emit governance.* outcomes)

Analytics Engine
→ Command UI (dashboards + KPIs)

Canonical System Loop (One Line)

Engines write state → Signal records events → Pulse automates → Zero governs → Analytics measures → UI acts

---

What This Map Enables

predictable development boundaries

clean multi-agent work without collisions

safe governance gating for risky actions

extension integration through signals rather than hacks